



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

DOOM rulat pe ESP32-S3 cu input prin bluetooth

Proiectare cu MicroProcesoare

Asistent de laborator: Andrei Diaconescu

Student: Stoica Mihai-Nicusor

Grupa: 30237

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

2 Decembrie 2025

Cuprins

1	Descriere	2
2	Componente Hardware	3
2.1	Unitate Principala (Game Engine)	3
2.2	Unitate Secundara (I/O Controller)	3
2.3	Tabel Conexiuni (Pinout)	3
3	Schema Sistemului	4
4	How to Use (Mod de Utilizare)	5
4.1	Configurare Initiala	5
4.2	Conectare Controller	5
4.3	Controale in Joc	5
4.4	Ecran Secundar	5
5	How to Do (Detalii Implementare)	6
5.1	Configurare ESP-IDF (Pentru ESP32-S3)	6
5.2	Partitionare Flash (Storage)	6
5.3	Arhitectura Software: I/O Controller	6
5.3.1	Mapare Input (Bluetooth la GPIO)	6
5.3.2	Receptie Stats prin UART	7
5.3.3	Display FSM	7
5.4	Modificari Software: Game Engine (ESP32-S3)	7
5.4.1	Maparea Software a Pinilor de Intrare	7
5.4.2	Sincronizare si Telemetrie	8
5.4.3	Optimizare Memorie: Dezactivare Audio	8
5.4.4	Probleme	8

1 Descriere

Acest proiect prezinta o implementare a jocului clasic DOOM pe un sistem embedded, bazat pe arhitectura microcontrollere-lor ESP32. Inovatia principala consta in utilizarea a doua unitati de procesare distincte care comunica:

1. Un **ESP32-S3** dedicat exclusiv motorului grafic si logicii jocului (Game Engine).
2. Un **ESP32 Standard** care actioneaza ca I/O Controller, gestionand conexiunea Bluetooth cu un DualShock 4 si un ecran secundar OLED pentru telemetrie.

Aceasta separare a responsabilitatilor permite rularea jocului la un framerate stabil, eliberand procesorul principal de sarcini costisitoare precum stiva Bluetooth si actualizarea display-ului secundar. Proiectul demonstreaza utilizarea eficienta a memoriei PSRAM externe, partitionarea customizata a memoriei flash pentru stocarea fisierelor WAD si comunicarea inter-chip prin UART.

2 Componente Hardware

Sistemul este alcatuit din urmatoarele module interconectate:

2.1 Unitate Principala (Game Engine)

- **Platforma:** ESP32-S3 (WROOM-1).
- **Resurse:** 240MHz Dual Core, 16MB Flash, 8MB PSRAM (Octal SPI).
- **Display Principal:** TFT LCD, driver ST7789, interfata SPI.

2.2 Unitate Secundara (I/O Controller)

- **Platforma:** ESP32 (Standard).
- **Resurse:** Bluetooth Classic/BLE suportat hardware.
- **Display Secundar (HUD):** OLED, driver SSD1306, interfata I2C.
- **Intrare:** Gamepad Sony DualShock 4 (conectat via Bluepad32).

2.3 Tabel Conexiuni (Pinout)

1. Conexiuni ESP32-S3 (Doom Engine):

Periferic	Pin (GPIO)	Funcție
TFT Display	13	MOSI (Data)
TFT Display	18	SCLK (Clock)
TFT Display	15	CS (Chip Select)
TFT Display	2	DC (Data/Command)
TFT Display	4	RST (Reset)
UART TX	17	Transmisie date (Health/Ammo)

2. Conexiuni ESP32 Standard (Controller):

Periferic	Pin (GPIO)	Funcție
OLED Display	27	SDA
OLED Display	26	SCL
UART TX	17	Transmisie Input catre S3
UART RX	16	Receptie Telemetrie de la S3
Simulare Butoane	18, 19, 21, 22...	Iesiri digitale mapate intern

3 Schema Sistemului

Diagrama de mai jos ilustreaza arhitectura dual-core distribuita. ESP32-ul secundar (dreapta) preia datele de la controller via Bluetooth si le trimite serial catre ESP32-S3 (stanga), care randeaza jocul si trimite inapoi starea jucatorului pentru a fi afisata pe OLED.

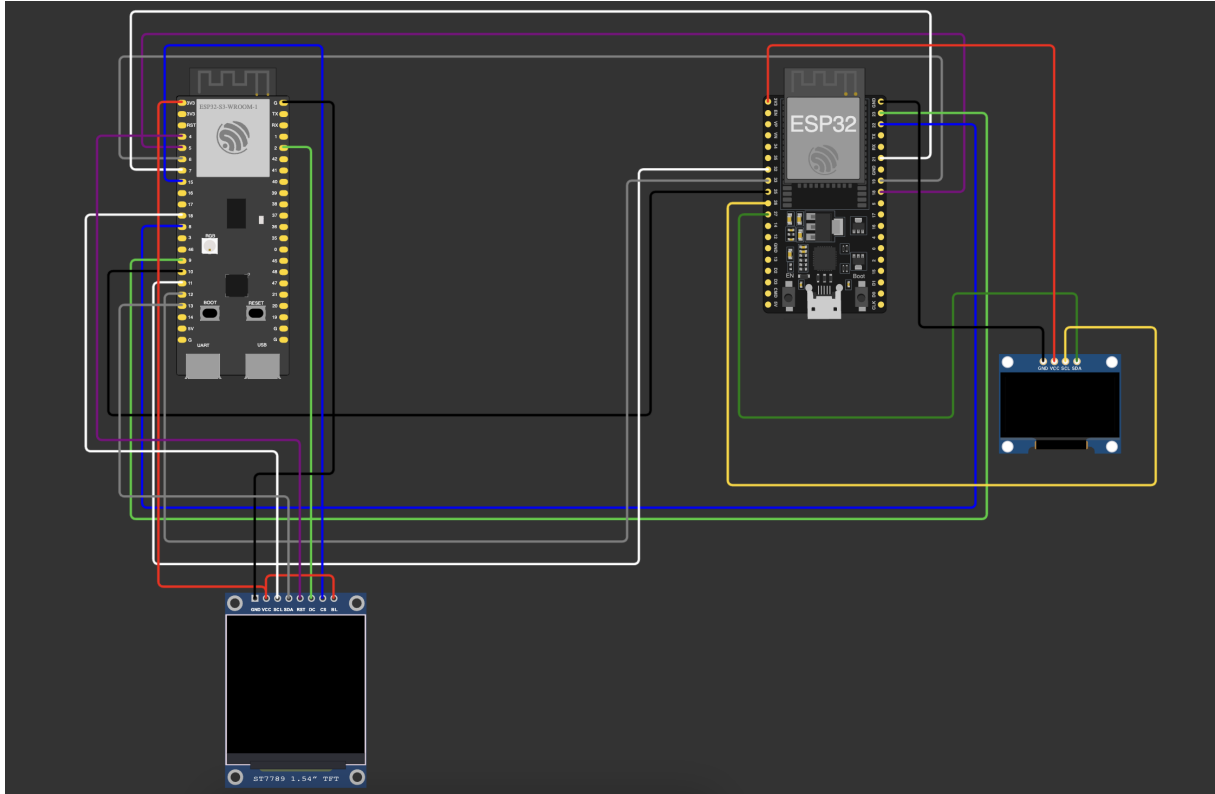


Figura 1: Schema electrica detaliata si fluxul de date

4 How to Use (Mod de Utilizare)

4.1 Configurare Initiala

1. Alimentati ambele placi ESP32 (S3 si Standard) la o sursa USB stabila.
2. Pe ecranul principal (TFT) va aparea secventa de boot DOOM.
3. Pe ecranul mic (OLED) va aparea mesajul: "Bluepad32 Init... Connect Gamepad...".

4.2 Conectare Controller

1. Porniti controllerul.
2. Activati modul de imperechere tinand apasat simultan butoanele **PS** si **SHARE** timp de 3 secunde (led-ul va clipi rapid).
3. ESP32-ul secundar se va conecta automat.
4. Pe OLED va aparea confirmarea: "Controller Connected".

4.3 Controale in Joc

Sistemul mapeaza automat butoanele controller-ului la actiunile din DOOM:

- **D-Pad** Miscare (W, A, S, D).
- **X:** Tragere (Fire / CTRL).
- **O:** Actiune/Deschidere usi (Use / Space).
- **Options:** Meniu / Enter.
- **Share:** Back / Escape.

4.4 Ecran Secundar

Urmariti ecranul OLED in timpul jocului pentru informatii critice:

- **Health:** Procentajul vietii ramase.
- **Ammo:** Munitia pentru arma curenta.

5 How to Do (Detalii Implementare)

Aceasta sectiune descrie pasii tehnici pentru reproducerea proiectului, configurarea mediului si, in detaliu, modificarile software aduse pentru a permite comunicarea complexa intre cele doua unitati.

5.1 Configurare ESP-IDF (Pentru ESP32-S3)

Jocul ruleaza pe framework-ul ESP-IDF. Fisierul `sdkconfig` este configurat astfel pentru performanta maxima:

- **Target:** `CONFIG_IDF_TARGET="esp32s3"`
- **Frecventa CPU:** Setata la maxim: `CONFIG_ESP_DEFAULT_CPU_FREQ_MHZ_240=y`
- **Memorie PSRAM:** Activata obligatoriu:

```
CONFIG_SPIRAM=y
CONFIG_SPIRAM_MODE_OCT=y
CONFIG_SPIRAM_SPEED_80M=y
```

- **Flash Speed:** `CONFIG_ESPTOOLPY_FLASHMODE_QIO=y` pentru citire rapida.

5.2 Partitionare Flash (Storage)

Fisierul `Doom1.WAD` (4MB) trebuie stocat in memoria flash. S-a folosit o tabela de partitii custom (`partitions.csv`) pentru a aloca spatiu suficient:

#	Name,	Type,	SubType,	Offset,	Size,	Flags
nvs,	data,	nvs,	,		0x6000,	
phy_init,	data,	phy,	,		0x1000,	
factory,	app,	factory,	,		1M,	
rofs,	data,	undefined,,			14M,	

Partitia `rofs` este montata ca un sistem de fisiere read-only, permitand motorului de joc sa acceseze resursele direct prin mapare in memorie.

5.3 Arhitectura Software: I/O Controller

Codul de pe ESP32-ul secundar (Standard) este scris in C++/Arduino si actioneaza ca un "Man-in-the-Middle" intre gamepad-ul Bluetooth si consola DOOM. Acesta indeplineste trei functii critice implementate astfel:

5.3.1 Mapare Input (Bluetooth la GPIO)

Deoarece ESP32-S3 se asteapta la semnale electrice fizice pe pinii sai de intrare, I/O Controller-ul traduce pachete Bluetooth in stari logice (LOW/HIGH). Biblioteca `Bluepad32` furnizeaza starea butoanelor, care este scrisa direct pe pinii de iesire conectati la S3:

```
1 // Functia controlGPIO() traduce pachetele in semnale electrice
2 void controlGPIO(ControllerPtr ctl) {
3     uint8_t dpad = ctl->dpad();
```

```

4 // Exemplu: Daca D-Pad UP este apasat, pinul devine LOW (Active Low)
5 digitalWrite(PIN_UP,      (dpad & DPAD_UP)      ? LOW : HIGH);
6 digitalWrite(PIN_FIRE,    ctl->a()              ? LOW : HIGH);
7 digitalWrite(PIN_USE,     ctl->b()              ? LOW : HIGH);
8 }

```

5.3.2 Receptie Stats prin UART

Pentru a afisa HP-ul si munitia, ESP32-S3 trimite date seriale. Am implementat un protocol binar strict cu un header de sincronizare 0xAA 0x55 pentru a preveni citirea datelor eronate (garbage data).

Structura pachetului de date (9 bytes total):

- **Header (2 Bytes):** 0xAA, 0x55
- **Payload (7 Bytes):**

```

typedef struct __attribute__((packed)) {
    uint8_t health;      // 0-100%
    uint16_t bullets;    // 2 bytes
    uint8_t shells;
    uint8_t rockets;
    uint16_t cells;
} GameDataPacket;

```

Algoritmul de citire verifica continuu semnatura de start:

```

1 if (Serial2.peek() != 0xAA) {
2     Serial2.read(); // Ignora octetii pana la gasirea header-ului
3     continue;
4 }
5 // Daca header-ul este valid, citim structura in memorie
6 Serial2.readBytes((char*)&gameData, sizeof(GameDataPacket));
7 newDataAvailable = true;

```

5.3.3 Display FSM

Ecranul OLED functioneaza pe baza unui FSM gestionat in bucla principala, asigurand feedback vizual constant:

- **SEARCHING:** Ruleaza animatia `drawConnectingScreen()` cand nu este detectat niciun controller.
- **WAITING:** Controller conectat, dar fara date de la joc (`drawWaitingForData`).
- **INGAME:** Afiseaza HUD-ul (`drawDoomHud`) cand `newDataAvailable` este true.

5.4 Modificari Software: Game Engine (ESP32-S3)

Pe unitatea principala (ESP32-S3), aplicatia este structurata in jurul functiei `mcugdx_main`, care orchestreaza initializarea perifericelor si bucla principala a jocului. Analizand codul sursa, se evidentiaza urmatoarele aspecte critice ale implementarii:

5.4.1 Maparea Software a Pinilor de Intrare

Desi ESP32-S3 nu comunica direct cu gamepad-ul, acesta primeste semnalele pe pinii GPIO, interpretandu-le ca apasari de taste. Configuratia definita in cod este urmatoarea:

GPIO Pin	Tasta Doom (Actiune)	Descriere
5	KEY_FIRE (K)	Tragere
6	KEY_USE (L)	Deschidere Usi / Actiune
7	KEY_ESCAPE	Back / Menu
8	KEY_ENTER	Selectare
9	KEY_RIGHTARROW (D)	Rotire Dreapta
10	KEY_DOWNARROW (S)	Mers Inapoi
11	KEY_LEFTARROW (A)	Rotire Stanga
12	KEY_UPARROW (W)	Mers Inainte

Tabela 1: Maparea pinilor GPIO la tastele interne ale motorului Doom

5.4.2 Sincronizare si Telemetrie

Bucula principala (`DG_DrawFrame`) contine logica de trimitere a datelor catre ecranul secundar. Pentru a nu satura magistrala UART si a mentine performanta randarii grafice, pachetele de date (Health, Ammo) sunt trimise doar daca au trecut cel putin **50ms** de la ultima transmisie. Acest mecanism asigura un echilibru intre rata de actualizare a HUD-ului si FPS-ul jocului.

5.4.3 Optimizare Memorie: Dezactivare Audio

Un detaliu tehnic crucial este gestionarea sunetului. In codul sursa, functia `mcugdx_sound_load_raw` este definita ca un *stub* (returneaza `NULL`), iar functiile de caching audio sunt dezactivate. Aceasta decizie a fost luata pentru a evita erorile de tip *Out of Memory* si fragmentarea heap-ului, probleme frecvente pe ESP32-S3 cand se incearca incarcarea simultana a texturilor grafice si a sample-urilor audio in PSRAM. Astfel, stabilitatea vizuala a fost prioritizata in detrimentul efectelor sonore.

5.4.4 Probleme

Momentan singura variabila de telemetrie ce este transmisa cu succes pe al doilea display este cea de health. In unele momente se intampla ca jocul sa dea crash atunci cand se umple frame bufferul. In acest caz ESP-ul se reseteaza automat, dar uneori ajuta un nou flash pentru a elibera complet PSRAM-ul. Nu se stie cauza exacta pentru care frame buffer-ul da overflow.

Referințe

- [1] KOTerra, *utcn-DOOM-ESP32S3-dual-screen Repository*, GitHub, 2025.
<https://github.com/KOTerra/utcn-DOOM-ESP32S3-dual-screen>
- [2] Mario Zechner, *mcugdx - Cross platform game development for microcontrollers*, GitHub.
<https://github.com/badlogic/mcugdx>
- [3] Ricardo Quesada, *Bluepad32 - Bluetooth gamepad support for ESP32*, GitHub.